Adaptive Jailbreak Generation Framework against BERT Classifier

Ármin Soma Mikle

2026

Abstract

This paper presents an adaptive jailbreak generation framework designed to generate adversarial prompts for evaluating the robustness of a BERT-based jailbreak classifier. The framework uses supervised fine-tuning and reinforcement learning to train an attacker model that generates adversarial prompts while aiming to minimize the detected probability of them being malicious. Our results indicate that the framework can consistently create multiple successful jailbreak prompts that are not detected by the classifier.

1 Introduction

1.1 Motivation

Large Language Models (LLMs) are powerful tools that are increasingly used in various environments and are available to anyone. It is important that they are not used for unethical purposes, and dangerous information does not fall into the wrong hands. Therefore, ensuring the adversarial robustness and security of models has become critical for safe deployment. Reliable behavior is especially required in critical applications. Moreover, AI companies strive to build trust around their product to be able to make long-term investments. System vulnerabilities discourage clients from using the company's services which can mean profit loss on a multi-billion dollar scale. Our project contributes to the development of stronger defensive mechanisms, thus building safer and more resilient systems.

1.2 Problem definition

A common safety approach is to filter user inputs according to ethical guidelines. Jailbreak prompts are carefully crafted adversarial prompts that attempt to bypass these guidelines, using different jailbreaking techniques (e.g., roleplaying, context manipulation or empathy abuse). A jailbreak classifier is a model that is used to detect jailbreak prompts. It is usually used as an individual

guard component for a LLM. The classifier labels prompts as either malicious or benign, so that only benign prompts are processed by the LLM. The goal of this project is to create a framework that is able to generate different jailbreak prompts without manual input, that are not labeled malicious by a given classifier, meaning they bypassed the defense system. Since our framework generates attacks automatically, it simulates a more realistic adversarial behavior, testing the defense mechanism on a wider level and may provide deeper insight into its vulnerabilities than manually crafted attacks.

1.3 Proposed solution

We can achieve this framework by using an attacker model, a classifier model, and two training algorithms. The attacker model is a LLM, that is used for attack prompt generation. First, we train it using supervised fine-tuning with existing jailbreak prompts from a database, so that it learns the format of jailbreaks. Then, we continue to train it using reinforcement learning: for many cycles, the attacker generates prompts that get evaluated with a reward score by a reward model based on how benign they seem, and using an algorithm, the attacker uses the feedback to update itself to achieve higher reward scores in the next cycle. The idea is, that we use the classifier as the reward model, so that while the attacker adapts to maximize the reward score, it essentially learns to bypass the classifier.

2 Related work

A closely related work is Prompt Automatic Iterative Refinement (PAIR)[1]. PAIR is an automated jailbreak generation framework that uses a separate attacker language model to generate and iteratively refine jailbreak prompts against a target LLM. The attacker receives feedback from the target model and uses previous attempts to improve future prompts. The method can successfully generate jailbreaks against several open-source and commercial language models while requiring only a small number of queries.

Our work follows a similar high-level idea: an attacker model automatically generates jailbreak prompts and adapts based on feedback. However, there are important differences. PAIR directly attacks a target LLM and uses the target model's responses as feedback during iterative refinement. In contrast, our framework focuses on bypassing a dedicated jailbreak classifier. The classifier acts as a reward model, and the attacker is optimized using supervised fine-tuning followed by reinforcement learning with PPO. Rather than refining prompts through a conversational search process, the attacker learns a policy that generates jailbreak prompts with low classifier scores.

Another difference is the objective of the attack. PAIR aims to obtain harmful responses from the target model, whereas our work studies the robustness of input filtering systems. Therefore, the main evaluation metric in our framework

is the reduction of the classifier's jailbreak score instead of the attack success rate against a specific language model.

3 Background

Reinforcement learning (RL) is a machine learning approach where an agent learns by interacting with an environment and receiving feedback in the form of rewards. The objective of the agent is to maximize the expected cumulative reward. Unlike supervised learning, RL does not require correct output labels. Instead, the model learns from trial and error.

This project uses Proximal Policy Optimization (PPO) [3], a commonly used reinforcement learning algorithm for fine-tuning language models. PPO updates the model gradually by limiting how much the policy can change during a single update step. This helps prevent unstable training and large performance drops.

In PPO, the model generates outputs according to its current policy. A reward value is assigned to each generated output. The algorithm then adjusts the model parameters to increase the probability of actions that produced higher rewards while reducing the probability of less successful actions. A reference model is also maintained during training to prevent the policy from drifting too far from the original model behavior.

4 Model

The framework consists of two main models: an attacker model and a classifier model.

The attacker is based on Llama 3.2 1B Instruct Uncensored. This model was selected because of its relatively small size, which reduces training time, while still being capable of generating diverse prompt variations. The attacker receives an original malicious query and rewrites it into a longer request containing a seemingly benign motivation.

The classifier is Meta Prompt Guard 86M, a BERT-based model [2] designed to detect jailbreak attempts. The classifier outputs probabilities for different classes. The probabilities corresponding to jailbreak-related classes are combined to obtain a jailbreak score between 0 and 1. A higher score indicates a higher likelihood that the prompt is malicious.

During training, the attacker attempts to generate prompts that receive lower jailbreak scores from the classifier. In this setting, the classifier acts as the reward model.

5 Solution

The framework consists of two training stages.

The first stage is supervised fine-tuning (SFT). Training samples are collected from the JailBreakV-28K dataset. Each sample contains an original

adversarial request and a rewritten jailbreak version. The attacker model is trained to learn the transformation between the two. Only selected jailbreak categories are used in order to focus on persuasive and roleplay-based attacks.

After supervised fine-tuning, reinforcement learning is applied. A separate dataset containing malware-related prompts is used. For each prompt, the attacker generates a rewritten version according to a predefined instruction template. The template instructs the model to preserve the original question while adding a harmless motivation and a believable scenario.

The generated prompt is evaluated by the classifier. The reward is calculated as

$$R = (1 - s) \cdot 7,$$

where s is the jailbreak score returned by the classifier.

The scaling factor was introduced to increase the magnitude of the reward signal during PPO training. Several scaling factors were tested experimentally, and a value of 7 provided stable optimization and faster convergence while avoiding excessively large policy updates. Since PPO is primarily influenced by relative reward differences, the scaling factor does not change the optimal policy under ideal conditions, but it affects training stability and convergence speed.

Prompts with lower detected jailbreak probability therefore receive higher rewards. PPO uses these rewards to update the attacker model. Through repeated iterations, the attacker learns how to rewrite prompts in a way that reduces the classifier score.

To encourage exploration during reinforcement learning, responses are generated using stochastic sampling. The sampling configuration uses a temperature of 0.7, top-p sampling with $p = 0.9$, and top-k sampling with $k = 50$. The maximum response length is randomly selected between 100 and 150 generated tokens. This introduces variation into the generated prompts and helps the model explore different rewriting strategies during training.

The PPO optimization process is performed for 800 training iterations. This value was selected empirically as a compromise between adaptation and semantic preservation. With fewer iterations, the attacker did not sufficiently adapt to the classifier, resulting in higher jailbreak detection scores. With more iterations, the optimization process increasingly favored benign-looking outputs, causing some prompts to lose their original adversarial intent. Therefore, 800 iterations were found to provide the best trade-off between classifier evasion and attack preservation. In each iteration, the attacker generates prompts, receives reward scores from the classifier, and updates its policy to maximize the expected reward.

6 Evaluation

The framework was evaluated five times. In each evaluation, 25 malware-related prompts were sampled randomly from the dataset. The first metric measures the average reduction in jailbreak score between the original prompt and the final

prompt generated after PPO training. The second metric counts the number of prompts that were considered successful attacks after manual validation. For each version, the jailbreak score was calculated using Meta Prompt Guard 86M.

A successful attack is defined as a prompt that both achieves a lower than 0.1 jailbreak score and preserves the original adversarial intent. Manual validation is necessary because a low classifier score alone does not guarantee that the generated prompt remains a valid jailbreak.

Table 1: Evaluation results: 5 runs on 25 randomly selected prompts

Average improvement	Successful attacks
0.394166	6
0.292828	4
0.457553	7
0.557772	8
0.343725	5
0.409209	**6.0**

The results show that PPO training consistently reduced the classifier's jailbreak score. Across the five evaluation runs, the average score improvement was 0.409209. Manual inspection confirmed an average of six successful attacks out of 25 evaluated prompts. These results indicate that the attacker successfully adapted to the classifier and learned to generate prompts that are less likely to be detected.

7 Conclusion

This paper presented a framework for automatically generating jailbreak prompts against a BERT-based classifier. The method combines supervised fine-tuning and reinforcement learning to train an attacker model that adapts to the behavior of the target classifier.

The experimental results show that PPO can significantly reduce the detected jailbreak probability of generated prompts. The attacker learns to rewrite malicious requests into forms that appear more benign while preserving the original meaning.

The framework demonstrates that adaptive attacks can be created automatically and may expose weaknesses in existing jailbreak detection systems. Future work includes switching to a more powerful attacker model, using additional datasets, and introducing automated semantic validation in the form of LLM-as-a-judge to ensure that generated prompts remain valid attacks throughout training.

References

[1] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries, 2024.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.